A self-interpreter for SKI written in SKI: Extensional semantics for new symbols

N. E. DAVIS

ZeroAttest, United States of America
(email: neal.davis@protonmail.com*)*

Abstract

We author a metacircular self-interpreter for the SKI combinatory calculus, explicitly introducing symbols within the object language. This interpreter is represented as a closed term of the basis $\{S, K, I\}$ (three leaves plus application) weighing in at 618 atoms, which runs on an encoding of itself in 504,930 contractions. Extending the object language with a fifth symbol (a leaf with no contraction rule) forces the interpreter's result type to grow, and whether the new outcome is exposed as a value or absorbed into divergence is decided by a single arm of the interpreter. The same site, made a parameter, becomes a nonlocal lookup in the style of a peek into an append-only namespace: the resolver is passed as an argument, a lookup may block, and a namespace of answers accumulates across re-runs, keyed by a 240-atom structural equality check on encoded paths. The loop is fueled, and we prove it adequate: a budget of k admits exactly the terms reaching weak head normal form in fewer than k contractions, so the least adequate budget is one more than the number of contractions and the answer is otherwise independent of the budget. Two further structural results frame the construction. A census over the $\{B, C, K, W\}$ basis shows that representation needs weakening, that iteration needs contraction, and that contraction alone already costs normalization. Finally, no SKI term quotes *every* live term; on the encoded side of that boundary everything, including metacircular evaluation, is closed.

1 Introduction

Metacircular evaluation permits the separation of the evaluation mechanism from the object language, allowing for controlled experimentation with different evaluation strategies and the observation of intermediate states without affecting the host environment. An interpreter can be employed to handle incomplete evaluation without interfering with the host environment, particularly in cases in which the host environment itself may not be capable of representations such as partially reduced terms, intermediate representations of the object language, crashes, and other exceptional states.

We construct a symbolic step-evaluator for the SKI combinator calculus as a closed term of the basis $\{S, K, I\}$. This evaluator performs symbolic evaluation by repeatedly applying the reduction rules of the SKI calculus until a normal form is reached. Because the interpreter is capable of symbolic representation beyond simply the basis $\{S, K, I\}$, we can add in functionality such as handling additional combinators, supporting custom reduction strategies, and managing exceptional states like crashes, undefined terms, or as-yet-unknown referentially transparent lookups.

The current line of research is motivated by the question of how amenable the SKI combinator calculus is to metacircular evaluation, and what design patterns and implementation techniques could facilitate such evaluation. We do not anticipate such a line of inquiry to



© 2026 Copyright held by the owner/author(s).
This work is licensed under a Creative Commons Attribution 4.0 International License.

be competitively efficient versus other combinator systems or higher-level representations, but rather as a tool for further exploration of the metacircular evaluation process and its possibilities for simple programs. This paper makes three contributions:

(1) A self-interpreter for $\{S, K, I\}$ in $\{S, K, I\}$. This evaluator is a step-evaluator over encoded terms rather than a decoder, together with the structural census that says what each rule of the calculus is bought for (§3, §4).
(2) The observation that extending the object language forces the interpreter's outcome type to grow, and that the meaning of the new outcome is authored at one site, with a lemma that nothing else changes (§5).
(3) Site as parameter. This is a value lookup whose resolver is an argument, which can block, and whose namespace of answers grows across re-runs over structurally compared paths (§6).

Section 2 places the work in the metacircular tradition; §7 reads the construction against virtualization in the Nock combinator calculus. The artifact and its reproduction are described in Appendix A.

2 Metacircular evaluation and self-interpretation

While many languages nominally support it, metacircular evaluation as a first-class strategy has only occasionally been applied outside of the Lisp family, where it has been used to implement features such as macro systems, partial evaluation, and sandboxed execution environments. Essentially, each nested interpreter acts as a virtual machine for the next level of interpretation. Lisp's homoiconic nature is particularly apt for such virtualization, since the interpreter can directly treat its interpretand as data, enabling Lisp's powerful reflective and metaprogramming capabilities. (In this connexion, the collapsing towers of Amin and Rompf (2018) will be of special interest.)

The lineage runs from McCarthy (1960), whose universal function `eval` was written in the language it evaluates, and in particular from the Lisp 1.5 manual (McCarthy et al. 1962), which specifies the language by that function. Reynolds (1972) gave the form its critical treatment: a definitional interpreter written in the style of its own object language is metacircular; the interpreted language inherits the defining language's evaluation order unless that order is made explicit. (The present work meets that observation directly in §5.1, where the object-level laziness of K is inherited rather than implemented.) Guy L. Steele and Sussman (1978) and Abelson et al. (1996) made the metacircular evaluator a standard instrument for experimenting with semantics, the very use to which we put it.

The Nock combinator calculus (est. 2002–2006) explicitly adopted self-application as a fundamental mechanism for metacircular evaluation from its earliest inception (Yarvin 2018). This mechanism was originally intended as a "hyper-Turing" operator (Davis 2025), in which a query for a referentially transparent value could answer, refuse permanently, or defer.

The reflective tower as such was introduced by Smith (1984); their 3-Lisp runs each level of a reflective tower on an interpreter at the level above. Wand and Friedman (1988) gave the tower a non-reflective description, and Asai et al. (1996) defined the language Black (in whose style Amin and Rompf's Purple is cast). Amin and Rompf (2018) treated the tower itself as the object of study. Their multi-level λ-calculus provides staging constructs and stage polymorphism, so that a single evaluator acts as an interpreter or as a compiler depending

on runtime parameters, and their metacircular evaluator Pink collapses arbitrarily many levels of self-interpretation into one compiled program; correctness of self-interpretation and multi-level Jones optimality are stated without formal proof but backed by experiment (their Propositions 4.2 and 4.6). Semantic modifications introduced at any level of the tower, such as instrumentation, CPS conversion, or reflection in the style of Black, survive the collapse. Their objective was performance, the removal of interpretive overhead, and their semantics were value-producing throughout: there is no fuel, no failure, and no outcome that has to be represented rather than computed. The present work complements their approach. We retain the levels with payment (§7), and we ask what an interpreter does in the case where its object language gains a construct whose meaning the interpreter must author (§5); this question does not arise in a setting without a third outcome (that is, beyond the value or divergence $\bot$). The two share the shape of the mechanism: an interpreter as an ordinary program in the calculus it interprets, with one place at which its behaviour is substituted. §6 builds upon this foundation.

In the λ-calculus proper, Barendregt (1991) stated the requirement on a self-interpreter, $\mathbf{E}\ulcorner M\urcorner = M$, and noted that the coding $\ulcorner\cdot\urcorner$ is a metalevel operation rather than a term; that boundary is located for SKI by a theorem of Jay and Given-Wilson (2011) (§3.3). Berarducci and Böhm (1993) showed the choice of encoding to be load-bearing, obtaining a self-interpreter which has a normal form. Brown and Palsberg (2016) carried self-interpretation into System F_ω, in which typing rather than quotation forms the barrier. Futamura's first projection (Futamura 1971) handles the specialization of an interpreter to a fixed program. Jones et al. (1993) provided the standard account of the subject. Jones (1996) identified the optimality condition which must be met to remove interpretive overhead entirely.

Self-interpretation for a Turing-complete system is as old as universality: Kleene (1936) gave the normal form theorem from which a universal function follows, and Turing (1936) the universal machine. Mogensen (1992) built a compact self-interpreter and self-reducer in the untyped λ-calculus; Jay and Palsberg (2011) built a typed one, in a combinatory calculus with a factorisation operator though not in $\{S, K\}$. (In particular, their distinction between self-*recognisers*, which recover a term from its representation, and self-*enactors*, which execute it, is necessary here.) The closest prior construction for SK is that of Tromp (2006), whose Theorem 3 gives a λ-term F of size 124 with $F\,C\,(\tilde{M}\ :\ N) = C\,M\,N$ for a bit-string encoding $\tilde{M}$ of a combinator M, and whose Theorem 4 translates it into a 263-bit self-interpreter for combinatory logic; with $C = I$ it is a self-enactor. Architecturally F is a decoder: it reconstructs live syntax from bits and hands it to a continuation, after which reduction proceeds under the ambient calculus's own rules. It carries no object-level S or K rule in its body, and so cannot implement a step relation that differs from the host's, which is what we carry out in §5. The same paper contains a symbolic normal-form reducer over Scott-encoded de Bruijn terms for the λ-calculus, used only as a size constant.

Two features of the combinatory setting separate the current construction from its λ-calculus relatives. The first is a simplification: with no binders there is no α-equivalence, no capture and no de Bruijn apparatus, so an encoding of syntax is an encoding of a binary tree over three leaves and nothing more. Much of what Mogensen's representation manages away does not arise here. The second, obstruction, pulls in the other direction. In the λ-calculus, quotation of a live term is unavailable because the coding is a metalevel operation (Barendregt 1991); here *factorisation* of a live term is not representable at

all (Jay and Given-Wilson 2011), so a term cannot discover whether its argument is an application, let alone recover its parts. What a λ-calculus self-interpreter may take from the structure of what it is handed, an SKI interpreter must be given as data. The construction here is a step relation over an encoding rather than a decoder in the manner of Tromp (2006), which facilitates the authored site of §5: once the step relation is data, a rule may be written that the host does not have.

Each ingredient of this construction has a precedent. A symbolic step-evaluator for SKI in the λ-calculus is a textbook exercise in a computer science curriculum, and bracket abstraction carries it into $\{S, K, I\}$ mechanically; censuses of combinator bases are old; fuel is the standard device for making divergence observable. What we have not found elsewhere is the combination. Our interpreter is a concrete closed term of 618 atoms whose step rules are data rather than re-spliced host syntax, which permits a rule the host does not have to be written at all. With this in hand, the authorship of a symbol's semantics localizes to a single arm (Lemma 5.3). Making the same arm into a parameter turns the interpreter into a namespace with blocking and replay (§6). A census (§4) locates what each of these facilities costs in the carrying calculus. Amin and Rompf's value domain consists of four constructors, all of them values (Amin and Rompf 2018, p. 6); a mismatched elimination form is a metalanguage error, not an outcome their semantics represents. A closed S, K, I term has no metalanguage to fail into, so every outcome it admits must be represented in the object language—first the timeout made observable by fuel (§5.1), then, once the object language is extended, a failure (§5). Jay and Given-Wilson's result concerns the limitations of SKI over live terms rather than what an interpreter may choose to expose; it is what bounds reification here (§3.3).

3 The interpreter

3.1 *Encoding and construction*

The $\{S, K, I\}$ terms are the object language of the interpreter; the standard definitions are:

$$I = \lambda x.x$$
$$K = \lambda x.\lambda y.x$$
$$S = \lambda x.\lambda y.\lambda z.x\ z\ (y\ z)$$

Object terms are represented by Scott encoding (Jansen 2013; Scott 1962), in which a datum of an inductive type is a function that takes one continuation per constructor and applies the continuation for its own constructor to its own components. The type of object terms has four constructors, three leaves and application, so an encoded term takes four continuations, in the fixed order S, K, I, application:

$$\langle S \rangle = \lambda s.\lambda k.\lambda i.\lambda z.\ s$$
$$\langle K \rangle = \lambda s.\lambda k.\lambda i.\lambda z.\ k$$
$$\langle I \rangle = \lambda s.\lambda k.\lambda i.\lambda z.\ i$$
$$\langle t\ u \rangle = \lambda s.\lambda k.\lambda i.\lambda z.\ z\ \langle t \rangle\ \langle u \rangle$$

A leaf discards all four continuations but its own. An application applies the fourth continuation to the encodings of its two children; this is how the interpreter walks a term, by applying $\langle t \rangle$ to four continuations as a case analysis on the head constructor of t. For example, $\langle K \rangle$ applied to four distinct atoms $X_1\ X_2\ X_3\ X_4$ reduces to X_2, and $\langle I\ K \rangle$ applied to the same four

reduces to $X_4\,\langle I\rangle\,\langle K\rangle$. Bracket-abstracted into $\{S, K, I\}$, the encodings are closed combinators:

$$\langle K\rangle \mapsto K\,(S\,(K\,K)\,K);$$
$$\langle I\rangle \mapsto K\,(K\,K);\text{ and}$$
$$\langle S\rangle \mapsto S\,(K\,K)\,(S\,(K\,K)\,K).$$

$\langle I\,K\rangle$ is a 40-atom term which bears some discussion. The bracket abstraction of $\langle I\,K\rangle = \lambda s.\lambda k.\lambda i.\lambda z.\ z\,\langle I\rangle\,\langle K\rangle$ reads as three components applied together: the application constructor encA $= \lambda t.\lambda u.\lambda s.\lambda k.\lambda i.\lambda z.\ z\,t\,u$, 32 atoms, applied to $\langle I\rangle$ and then to $\langle K\rangle$. Within the constructor, the three leading prefixes $S\,(K\,(S\,(K\,K)))$ are the abstraction pushing the three unused binders s, k, i past the body, and the inner part applies z to t and u. Written out (with line breaks carrying no meaning),

```
S (K (S (K K))) (S (K (S (K K))) (S (K (S (K K)))
  (S (S (K S) (S (K K) (S (K S) (S (K (S I)) K)))) (K K))))
(K (K K))
(K (S (K K) K))
```

where the first two lines are `encA`, the third is $\langle I\rangle$, and the fourth is $\langle K\rangle$; the term is drawn in Figure 5. Applied to any four terms, it hands $K\,(K\,K)$ and $K\,(S\,(K\,K)\,K)$ to the fourth of them.

The encoding $\langle\cdot\rangle$ itself is a metalevel operation, for the reason given in §3.3. Lists, the `Maybe` of the result type, and the fuel numeral are Scott-encoded the same way:

- `nil` $= \lambda n.\lambda c.\ n$;
- `cons` $h\ t = \lambda n.\lambda c.\ c\,h\,t$;
- `nothing` $= \lambda n.\lambda j.\ n$;
- `just` $x = \lambda n.\lambda j.\ j\,x$;
- `zero` $= \lambda z.\lambda s.\ z$; and
- `suc` $n = \lambda z.\lambda s.\ s\,n$.

Our definitions below are written with named binders and expanded to a closed $\{S, K, I\}$ term by bracket abstraction, with recursion through the fixpoint combinator Y. The constructed interpreter, called `whnfF`, consists of a 618-atom term; its unquoter `UQ`, which maps encoded data back to a live term, contains 43 atoms. The components of `whnfF` are given in Figure 1, with the full term given in Appendix A and depicted graphically in Figure 4. An evaluation is illustrated in Figure 2.

`whnfF` defines a fueled interpreter that evaluates encoded terms to weak head normal form. The fuel loop `whnfF n` $\langle t\rangle$ peels one `suc` from the numeral per step-attempt and returns `nothing` when the numeral is exhausted. Each attempt calls `step`, which walks the spine of the encoded term to its head leaf, collecting the arguments into a list, and dispatches on the leaf: the arm for I needs one argument, for K two, for S three. A saturated arm performs one contraction as data, building $\langle x\,z\,(y\,z)\rangle$ in the case of S, and the rebuilder `rb` re-applies the untouched arguments; the loop continues on the result with the remaining fuel. An unsaturated head means the term is in weak head normal form, and the loop returns it as `just` $\langle t\rangle$. This is normal-order head reduction with one contraction per attempt, which matches the host reducer's count exactly.

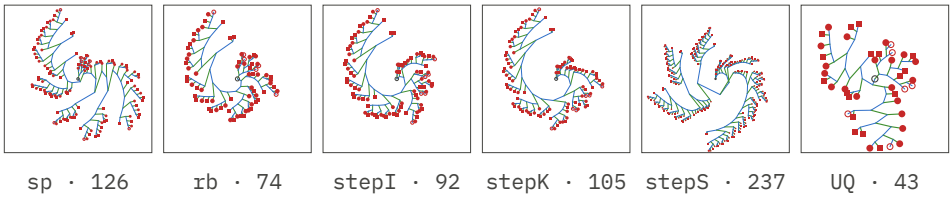


Fig. 1. The interpreter's components as terms, with every atom drawn: the spine walker `sp` (126 atoms), the rebuilder `rb` (74), the three step arms `stepI` (92), `stepK` (105), `stepS` (237), and the unquoter `UQ` (43). In this *every-atom* view a term is a disc: the root is at the centre, each leaf's angle is its in-order position (function subtree first, clockwise from the top), and its radius is its depth, so every atom of the term is visible. Function edges are green and argument edges blue; S is a filled circle, K a square, I a ring. Each panel is scaled to its frame, so sizes are not comparable across panels. The S arm is substantially larger because it builds $\langle x\ z\ (y\ z)\rangle$ as data.

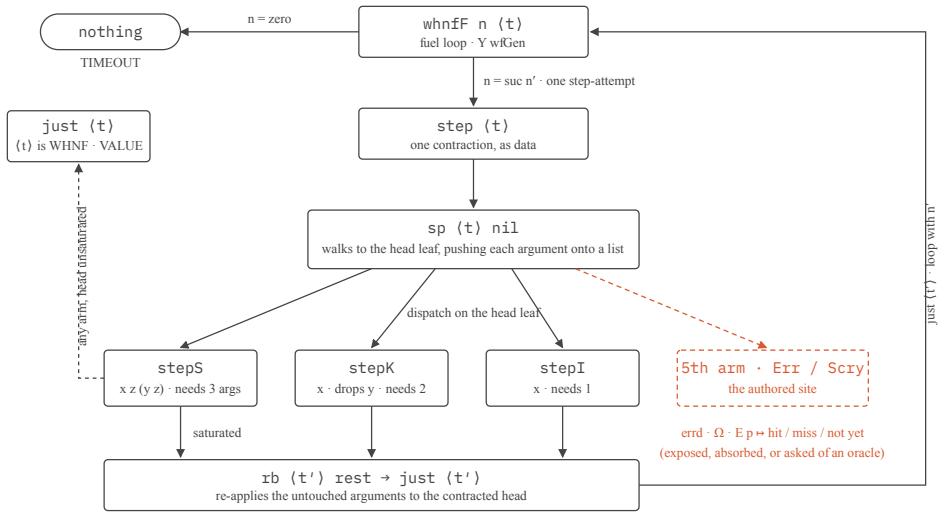


Fig. 2. The `whnfF` evaluation process. The fuel loop peels one `suc` per step-attempt; `step` walks the spine to the head leaf, collecting the arguments; dispatch selects the arm for that leaf; a saturated arm contracts and `rb` re-applies the untouched arguments, returning `just` $\langle t'\rangle$ to the loop; an unsaturated head returns `just` $\langle t\rangle$, the value; exhausted fuel returns `nothing`. The dashed fifth arm optionally contains an authored site (§5).

3.2 *Self-application*

The `whnfF` interpreter is applied to encoded terms to evaluate them to weak head normal form. This emulated process is not efficient: it incurs significant overhead due to the emulation of the host reduction steps. With fuel 3 on $\langle I\ K\rangle$ it reaches $\langle K\rangle$ in 340 host contractions; the decoded result equals the host's own reduction of $I\ K$. When it is run over the unquoter applied to an encoded term, it takes 91,556 steps. Run over an encoding of itself running on $\langle I\ K\rangle$, as a tower, it reaches weak head normal form in 504,930 contractions with the correct value. Empirically for this case, layer one costs 340 contractions to emulate one, and layer two

costs about 1,485 times layer one, the ordinary per-level overhead of naïve self-interpretation. Figure 3 shows the first run in progress and Figure 4 the interpreter as a term.

3.3 The encoding boundary

$S\,K\,K$ and I agree on every argument, yet they are distinct normal forms as terms of weak combinatory logic. A quotation function from live terms to encoded data must be able to distinguish them, which cannot be done by any pure SKI term.

Proposition 3.1. *No closed term Q over $\{S, K, I\}$ satisfies $Q\,M \twoheadrightarrow \langle M\rangle$ for every closed M.*

Proof. Jay and Given-Wilson (2011, Thm. 3.2) prove that factorisation of SK-combinators is a symbolic computation not representable by any SK-combinator; the proof is stated for $\{S, K\}$ and goes through unchanged with I primitive. Quotation would give factorisation: Q together with the unquoter `UQ` of §3.1 decomposes any term into its head and arguments by Scott dispatch on $\langle M\rangle$. Under the translation to the λ-calculus, $[\![Q]\!](\lambda x.x)$ is a single term, yet $Q\,(S\,K\,X)$ would yield $\langle S\,K\,X\rangle$, from which X is extractable, so every X would share a reduct, contradicting that S and K are distinct normal forms. □

Thus the encoder is a (better, *the only*) host function. Quotation is a one-time cost which must be paid to enter the representation. Everything on the other side of the boundary—the unquoter, the interpreter, their self-application, and the lookup machinery of §6 including equality on encoded paths—is ordinary $\{S, K, I\}$. Figure 5 shows the decoder beside a datum: in this calculus they are the same kind of thing.

4 The structural cost

What must any self-interpreter for SKI be able to do, structurally? S bundles two capabilities into one rule, composition and the duplication of its third argument, and never discards; K supplies discarding. The two capabilities that matter here, discarding and copying, coincide with the structural rules of weakening and contraction in substructural logic. By utilizing the basis $\{B, C, K, W\}$ for a comparative rewrite, we access the capabilities directly: B composes,

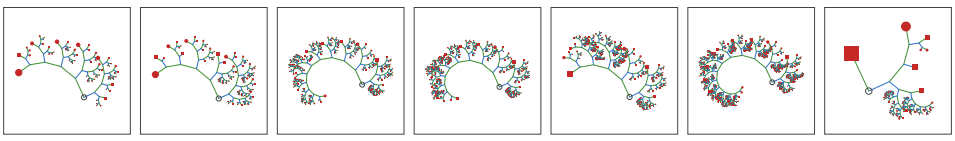


Fig. 3. The interpreter running `whnfF 3 ⟨I K⟩`, after 0, 20, 60, 120, 200, 300 and 340 host contractions. This is the *spiral* view, complementary to the every-atom discs of Figure 1: from the root, each application draws its function edge turning 25° left at full length and its argument edge turning 65° right at just over half length, so a long function spine curls into a spiral and arguments hang off it as diminishing fronds; subtrees below a length cutoff are pruned, so only the outermost atoms of a deep term are shown. It trades completeness for the global shape, which is what changes as the interpreter runs: the spine curls as Y unfolds and the encoded term is walked; the tree reaches about 111,000 atoms at step 300 and collapses to the answer, `just ⟨K⟩`, at step 340.

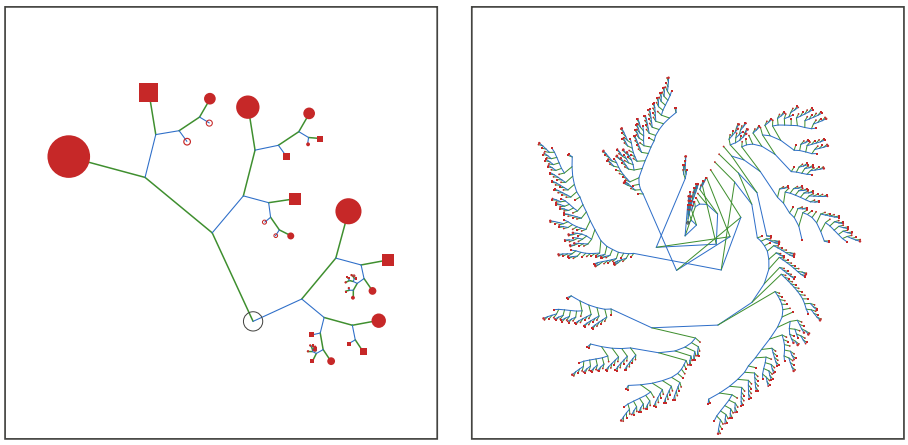


Fig. 4. The interpreter `whnfF` in both views. Left, the spiral (Figure 3), whose pruning leaves 48 of the 618 atoms visible. Right, the every-atom disc (Figure 1), 48 levels deep, in which each fern-like arm is one deeply nested argument as produced by bracket abstraction. Appendix A gives the term in full.

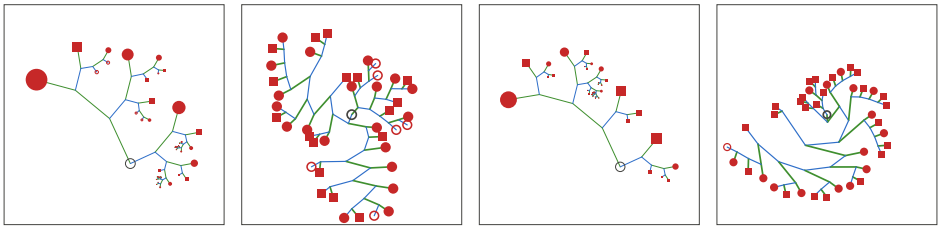


Fig. 5. The decoder and a datum. `UQ`, 43 atoms, and the Scott encoding $\langle I\,K\rangle$ of the two-atom term $I\,K$, 40 atoms, each in spiral and every-atom views. A datum and a program look alike because they are the same kind of thing; only the encoding step that produced $\langle I\,K\rangle$ lies outside the calculus.

C swaps, K discards an argument, and W duplicates an argument. We are interested in analyzing the structural cost of each component of the interpreter and the representation layer. Hence by rebuilding each component over the $\{B, C, K, W\}$ basis, we see which structural rule it requires, whether weakening (K) or contraction (W). We also count, independently of any abstraction algorithm, how often each binder occurs in each component's definition: a component needs weakening iff some binder occurs zero times, and contraction iff some binder occurs more than once. This occurrence census is the invariant measurement; atom counts are convention-relative (with I primitive the interpreter core shows seven W atoms, with $I = W\,K$ ten).

The representation layer is affine. All ten Scott constructors, the four term constructors together with `nil`, `cons`, `nothing`, `just`, `zero`, and `suc`, have no binder occurring twice: a

constructor selects one continuation and discards the rest, which is weakening by construction. The representation lives in BCK.

The interpreter core needs contraction at five of eight components. There are seven surplus occurrences in all: the unquoter's f (twice, in $(f\,t)(f\,u)$), the spine generator's accumulator (four times), the rebuild generator's head (twice), the S-arm's z (twice, in $x\,z\,(y\,z)$), the S rule itself reappearing one level up), and the fuel loop's term (twice). Any $BCKW$ abstraction must perform at least that many contractions.

Each rule is needed, and one of them costs normalization. Removing weakening removes dispatch: a Scott constructor is an n-ary discard. Removing contraction removes iteration: Y duplicates its argument, and the affine fragment BCK is strongly normalizing, so it has no fixpoint combinator and no fuel loop. (The fuel itself, a Scott numeral, is affine; iteration comes from Y.) The two are not symmetric. Contraction alone already admits divergence $\perp$: $\Omega = (S\,I\,I)(S\,I\,I)$ contains no K, and in $BCKW$ the term $W\,W\,W$ reduces to itself with no weakening anywhere, so the relevant fragment $BCIW$ is not normalizing while BCK is. Weakening is bought for representation and costs nothing in normalization; contraction is bought for iteration and is exactly what makes non-termination possible. Hence any self-interpreter for SKI must be written in a fragment where non-termination exists, and its outcome space cannot be $\{\text{value}\}$: it is at least $\{\text{value}, \perp\}$, which the next section makes observable by introducing fuel. The third outcome is not a design choice of this artifact but a consequence of self-interpretation.

The census also explains the artifact's shape. The representation is cheap because data only selects: every constructor and every case analysis is weakening, and the whole encoding of a term is built from discards. The cost is concentrated in five places, and all five are either recursion (the unquoter, the spine walker, the rebuilder, and the fuel loop each pass themselves or their state on twice) or the S rule reappearing one level up inside the arm that implements it, which is why that arm is the largest component (Figure 1). Two practical consequences follow: the representation layer can be carried unchanged into an affine calculus, with only the loop needing the full basis; and any effort to shrink the interpreter should be spent on those five sites, since everything else already bears the cost of a selection.

5 Extensional authored semantics

In a metacircular evaluator which augments interpretation, additional outcomes can be observed and handled explicitly, rather than being hidden as divergence. This permits partiality, evaluation that may not return a value, to be made explicit with concepts like fuel and `Maybe` in a basis that lacks them natively.

5.1 Fuel and the third outcome

An unfueled interpreter's outcome space is $\{\text{value}, \perp\}$. Divergence cannot be observed from inside a finite run, as fuel converts it to the observable set $\{\text{value}, \text{timeout}\}$. The limitation is local and bears a clear price: at fixed fuel k, one observation does not separate a divergent term from one that needs more than k steps. Ω times out at every k; $I^{31}K$ times out for $k \leq 31$ and yields a value from $k = 32$, since fuel k permits k step-attempts of which the last must be the no-redex check. Convergence is semi-decidable by raising k while divergence is not (the halting problem). Object-level laziness is inherited, so $KI\Omega$ discards Ω unevaluated under

the interpreter exactly as under host reduction because the object-level K rule fires before the interpreter recurses into the argument. (This follows Reynolds's observation.)

The off-by-one factor is an artifact of the interpreter's specification: write $M \Downarrow_n N$ when the closed term M reaches the weak head normal form N in exactly n normal-order contractions, and $M \Uparrow$ when it has no weak head normal form.

Lemma 5.1 (step adequacy). *Let M be closed over $\{S, K, I\}$. If M is in weak head normal form then* `step` $\langle M \rangle \twoheadrightarrow$ `nothing`. *Otherwise M has a head redex, and if $M \to_1 M'$ contracts it then* `step` $\langle M \rangle \twoheadrightarrow$ `just` $\langle M' \rangle$.

Proof. The spine walker `sp` decomposes $\langle M \rangle$ into a head leaf and an argument list. It recurses only on the application constructor, so it terminates on every encoded term, and it decomposes exactly as the host does. The head leaf then selects one of the three step arms, and that arm peels the argument list to the arity its rule requires. Since a term is in weak head normal form iff its head leaf is unsaturated, the arm finds too few arguments precisely when M is in weak head normal form; there it returns `nothing`, which is the first case. Otherwise the arity is present, the arm applies its rule to those arguments, and `rb` rebuilds the spine over the remainder. That result contracts the head redex, which is the leftmost one, and gives the second case. □

Theorem 5.2 (fuel adequacy). *Let M be closed over $\{S, K, I\}$ and let $k \in \mathbb{N}$.*

(1) If $M \Downarrow_n N$ and $n < k$, then `whnfF` $\langle k \rangle \langle M \rangle \twoheadrightarrow$ `just` $\langle N \rangle$.

(2) Otherwise if $M \Uparrow$ or $M \Downarrow_n N$ with $n \geq k$, then `whnfF` $\langle k \rangle \langle M \rangle \twoheadrightarrow$ `nothing`.

Proof. The loop is `loop n m = n nothing (loop1 loop m)` with `loop1 f m n2 = step m (just m) (f n2)`. Applying the Scott numeral $\langle k \rangle$ selects the first continuation when $k = 0$ and hands the predecessor to the second when $k = j + 1$.

Induct on k. For $k = 0$ the loop returns `nothing` immediately, and the hypothesis $n < 0$ of (1) is unsatisfiable, so (2) holds. For $k = j + 1$ the loop evaluates `step` $\langle M \rangle$ once. By Lemma 5.1 there are two cases. If M is in weak head normal form then `step` yields `nothing`, the first continuation of `loop1` is taken, and the result is `just` $\langle M \rangle$; here $M \Downarrow_0 M$ and $0 < k$, which is (1). Otherwise $M \to_1 M'$ and the result is `loop` $\langle j \rangle \langle M' \rangle$, to which the induction hypothesis applies: $M \Downarrow_n N$ iff $M' \Downarrow_{n-1} N$, and $n < j + 1$ iff $n - 1 < j$. □

We now state three consequences of these theorems. First, the least adequate budget for a converging M is exactly $n + 1$: one unit per contraction, and one for the no-redex check that establishes the answer is final. Second, the outcome is monotone in k, so a value once obtained is never withdrawn by more fuel — which is what makes searching for a budget by doubling sound. Third, the value is independent of k, so the interpreter's answer is a function of M alone and fuel is a resource bound rather than a parameter of the semantics. The theorems also fix the sense in which fuel is *not* neutral: n counts object contractions, and the host contractions the interpreter spends to simulate them are a separate quantity, measured in §4.

Note that fuel removes the interpreter's own divergence, not the object language's; §5.2 shows that an authored arm may reintroduce $\bot$ deliberately.

Table 1. Behavior of the two interpreters on selected object terms. The outcome "host diverges" refers to the host reducer's step cap being reached, not a detected non-termination generally; the arm is Ω.

object term	exposing (`errArmAbs`)	hiding (`errArmOmg`)
`Err`	ERR	host diverges
`Err K`	ERR	host diverges
`K I Err`	VAL	VAL
`I K`	VAL	VAL
Ω	TIME	TIME

5.2 *The site*

Extend the object language with a fifth constructor `Err`, with no object-level contraction rule. Its encoding is unremarkable, five continuations instead of four. When the spine walker finds `Err` in head position, the interpreter decides how to dispatch:

```
D("errArmAbs", ["acc"], a("errd"))  # observable: crash => value
D("errArmOmg", ["acc"], OMEGA)      # hidden: crash => host divergence
```

These arms are written $K\,(K\,(K\,I))$ and $K\,(S\,I\,I\,(S\,I\,I))$; our two full interpreters differ at this juncture. The result $\Omega = (S\,I\,I)(S\,I\,I)$ was selected as one of five divergent six-atom terms in this basis. The results of the two interpreters evaluating five different terms are summarized in Table 1.

Lemma 5.3 (Faithfulness at a site). *Let ℓ be a leaf of the object language and let E_1, E_2 be two interpreters over the same object type, identical except in the step arm selected for ℓ. For every object term whose reduction never brings ℓ into head position, E_1 and E_2 produce the same result.*

Proof. By Lemma 5.1 the spine walker selects the arm for ℓ only when the head leaf is ℓ, and selection is the only place either interpreter mentions that arm. On an input whose reduction never exposes ℓ in head position, every step either interpreter performs is taken in code the two share, so by induction on the number of steps the two sequences of intermediate terms coincide; by Theorem 5.2 the outcome is a function of that sequence and the budget. The hypothesis may be taken of either interpreter, since up to the first occurrence of ℓ in head position—if there is one—the two runs are the same run. □

Taking ℓ = `Err` and the two arms of §5.2 gives the pair this paper measures: the interpreters agree everywhere except where the authored symbol is actually reached, which is what makes the difference between them an observation about the symbol rather than about the interpreter.

The `K I Err` row of Table 1 refers to an instance in which `Err` is present but discarded by object-level laziness before it is ever in head position. Here it obliquely highlights that the addition of a constructor does not by itself introduce a new meaning: the meaning is authored downstream by the author of the interpreter.

6 Site as a parameter

The substitution as written in §5.2 affords a choice between two source variants. A lookup in the style of Nock's "scry" or peek into a bound referentially transparent namespace takes

the resolver as an argument instead. We build that in three steps, each extending the outcome type once more.

6.1 A synchronous resolver

The object language gains a fifth leaf `Scry`, of arity one, in a five-symbol encoding parallel to `Err`. The fueled loop gains a parameter: `wfQ E n` $\langle t \rangle$, where E is an ordinary encoded partial function from paths to values, a `Maybe`. When the spine walker reaches `Scry p` it applies E to p, the path exactly as written and unreduced. A hit splices the answer in and reduction continues. A miss is kept as a third step outcome distinct from "stuck", reusing the three-way type of §5.2, rather than folded into the existing `Maybe`, which would conflate "the resolver had no answer" with "no redex". The arm itself, in the same notation as the `Err` arms of §5.2, is

```
D("scHit",    ["rest","v"],
    A(a("stepped"), A(a("rbQ"), v("v"), v("rest"))))
D("stepScQ1", ["e","p","rest"], A(v("e"), v("p"),
    a("errd"),                      # nothing    -> the miss
    A(a("scHit"), v("rest"))))      # just v     -> splice and continue
```

that is, apply the resolver to the path and hand it two continuations: on `nothing`, the miss; on `just v`, splice `v` in and rebuild. The parameter is the resolver E; when E is fixed at compile time to one answer, the two arms become the `Err` arms of §5.2.

Against three resolvers (one that answers every path by echoing it, one that answers none, one that dispatches on the path's head leaf and answers only K), the loop matches the base interpreter on every term without `Scry`; hits and misses where expected; reports a bare `Scry` as stuck rather than as either; reduces `K K (Scry K)` to K without consulting the resolver, so a site inside a branch the program never forces is never consulted; and times out on Ω before any site is reached. Substitutability is not eager; since the path is unreduced, `Scry (I K)` and `Scry K` are different paths, a point to which §6.3 recurs. A path is not a sequence and not an address: it is whatever term occupies `Scry`'s argument position, delivered to the resolver as an encoded term, and it bears no meaning to the interpreter itself. Its interpretation is entirely the resolver's business. (Nock's scry paths are defined as lists of atoms; nothing here depends on such a shape.)

6.2 A blocking resolver

Since a real lookup can fail to answer for a given path (such as a network call or the result of a computation), a resolver that always answers is not a scry. Thus the resolver's type grows from two-way to three-way (`oJust`, `oNothing`, `oNotYet`) and the interpreter's outcome types grow to match, adding `pending` at the step level and `block` at the result level. The arm gains one continuation:

```
D("stepScN1", ["e","p","rest"], A(v("e"), v("p"),
    A(a("scHitN"), v("rest")),      # oJust v    -> splice and continue
    a("errdN"),                     # oNothing   -> the miss
    A(a("pendingN"), v("p"))))      # oNotYet    -> block, carrying the path
```

The difference against the synchronous arm above is the third line. This is the outcome-type growth of §5 again, triggered legibly as one continuation at one site.

Resuming a block captures no continuation. Because reduction is pure and deterministic, the driver re-runs the whole term from the top under a resolver that now also answers the missing fact. This is sound because the re-run's steps strictly before the block are identical to the first run's only if the new resolver agrees with the old one on every path the old one answered, which holds because the namespace is append-only. Re-running is not fuel-neutral: each round replays the previous one and then takes at least one more step, so with a fixed budget a block can become a timeout on the next round. Recomputation is charged for. The interpreter and each round's resolver are closed $\{S, K, I\}$ terms; the host side provides the fact table standing in for an external source of truth, the resume loop, and the detection of a path with no fact. (This gestures towards a desideratum of a host handler which can facilitate SKI evaluation in the context of blockable computations.)

The namespace is accumulative. With an empty namespace and facts $S \mapsto \texttt{Scry}\ K$, $K \mapsto I$, the term `Scry S` blocks on S, learns it, re-runs, and the newly substituted `Scry K` blocks on K, forced by the result of resolving the first, before the third round reaches the value I. A path with no fact blocks into a distinguishable stuck state, and fuel exhaustion remains distinguishable from a block.

6.3 *Compound paths*

The resolver of §6.2 dispatches on a path's head leaf only and cannot tell $S\,K$ from $S\,K\,K$. Proposition 3.1 does not force this, as that theorem concerns factorising *live* combinators. The path a resolver receives is Scott-encoded *data*, a tree with five leaf tags, and equality on such data is an ordinary structural recursion. As Jay and Given-Wilson said, once a term is encoded, factorisation "is a routine list operation". However, SK cannot produce the encoding from a live term.

`EQ5` is introduced as a structural equality test over the five-constructor encoding (Figure 6); and a resolver that compares the incoming path p against each known path in turn,

$$p \;\mapsto\; \texttt{EQ5}\ p\ \langle k_1\rangle\ (\texttt{oJust}\ \langle a_1\rangle)\ (\texttt{EQ5}\ p\ \langle k_2\rangle\ (\texttt{oJust}\ \langle a_2\rangle)\ (\cdots\ \texttt{oNotYet})),$$

which runs inside the blocking loop unchanged. This distinguishes a path from its prefix, resolving `Scry (S K K)` and `Scry (S K)` to different values under the same facts; it chains through a compound answer, `Scry (K I)` resolving to `Scry (S K)` and then to I in three rounds; and it refuses a false hit, blocking on `Scry (S K)` when the only fact is for $K\,S$.

Paths are intensional, syntactic objects, compared by structural equality of their encodings, exactly as a Nock scry path is a noun compared by noun equality. `Scry (I K)` blocks when the only fact is for K, and resolves when a fact is given for $I\,K$. Matching paths up to conversion constitutes the known-undecidable word problem for SKI; normalizing the path under fuel before lookup would be a fueled approximation thereto and would reintroduce the third outcome inside path resolution.

7 Virtualization and Nock

The Nock combinator calculus consists of a small set of primitive operations on nouns, which are the fundamental data structures in Nock. The details of Nock's construction are beyond the scope of this paper, but they can be found in the specification (Yarvin 2018). We are primarily interested in Nock due to its virtualization properties and the insights it provides into self-interpretation and substitution mechanisms.

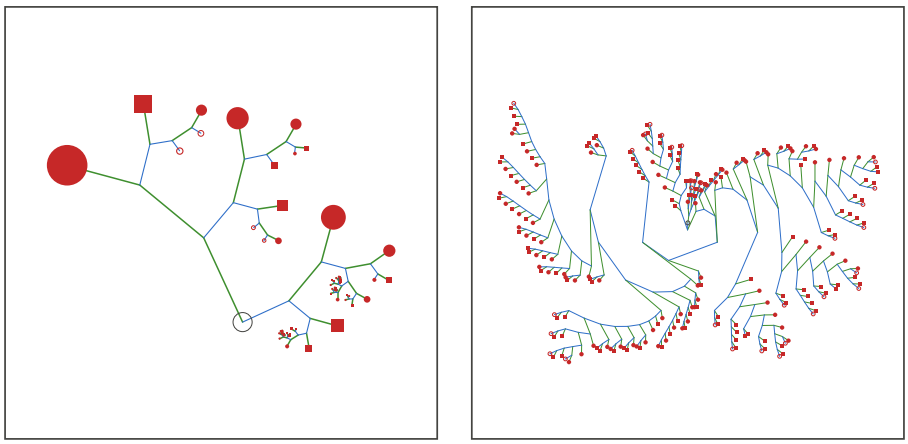


Fig. 6. `EQ5`, structural equality on encoded paths, 240 atoms: 92 S, 118 K, 30 I. Spiral view, 68 atoms visible, beside the every-atom disc.

Nock's virtualized interpreter `+mock` is interesting less because Nock can interpret Nock; that follows once a formula computes `nock`, since a formula already is a noun.; but instead in that a formula can be run through a *substituted* interpreter. Such an interpreter resolves a scry differently or catches a crash and returns it as a value, while agreement with the reference interpreter holds wherever the substitution is not exercised, and the substitution is confined to a named site. Read in this way, §5 is the SKI analogue of Nock virtualization. The tower demonstrates self-interpretation and fidelity, and a general version of that demonstration exists for value-producing towers (Amin and Rompf 2018, Props. 4.2 and 4.6). The two arms demonstrate the thing `+mock` is for: the same program, the same interpreter apart from one arm, different observable outcomes because that arm decides what one outcome category is allowed to be. Lemma 5.3 is the faithfulness requirement, virtualized with no substitution equals real, and the runs of §3.2 are its instances.

The analogy with Nock breaks at quotation. `+mock` never needs a quoting step, because a Nock formula already is the noun handed to the alternate interpreter. SKI is not homoiconic: S, K, I are opaque, and Proposition 3.1 states that no SKI term can turn an arbitrary SKI term into data about itself. SKI-on-SKI is therefore not symmetric the way Nock-on-Nock is. There is one unavoidable non-SKI quoting step, and virtualization is native only above that boundary and closed under further quoting.

Depth is bounded for an ordinary reason. Two levels already cost 504,930 contractions against a trivial base term, at roughly 10^3 per level; a third level was not attempted in this research program. (Nock-on-Nock virtualization has the same character, which is why runtime-matched accelerated code exists for Nock as a practical instruction set architecture. Notably, the reference runtime interpreter for Nock actually implements `+mock` and treats base Nock as a subset special case.)

8 Conclusion and open problems

An interpreter that is an ordinary term of the calculus it interprets makes the authorship of introduced meaning visible: a new symbol's semantics is a few atoms at one site, provably isolated from everything else; that same site made into a parameter furnishes a namespace. Our construction is small enough to be read whole (if by the superhuman; Appendix A) and cheap enough to extend, and several open research questions follow therefrom:

- *Partial evaluation.* A resolver that treats an unanswered path as an inert unknown, rather than as a block, turns the blocking loop into a partial evaluator: run over an encoding of the interpreter applied to a fixed program with the resolver unknown, it returns the program with every dispatch before the first lookup executed away, Futamura's first projection in its weakest form. The static prefix is saved exactly; what remains after the first dynamic dependency is the interpreter's continuation, and removal requires unfolding under an unknown head, which would be a separate project.
- *Quotation above level 0.* Proposition 3.1 bars quotation of live terms only. Re-encoding an already-encoded term, $\langle t \rangle \mapsto \langle\langle t \rangle\rangle$, is a data transformation and definable, and with it a tower can be deepened from inside; whether this recovers the self-quotation of Jay (2016, 2018) in the intensional setting is open.
- *An unquoter for the extended encoding,* and a pure-SKI driver for the blocking loop, would move the last host-side pieces of §6.2 into the calculus. The resolver's namespace is host-side only because its facts are compiled into a closure; an explicit core convention, a Scott pair of battery and payload in the manner of a Nock core, would make a fact an ordinary `cons` and the whole loop one closed term. That convention, and a subject-oriented surface language over it whose compile-time/runtime line for quotation is fixed by Proposition 3.1, are the subject of separate work. The semantics of intensional recursion (Kavvos 2017) is the natural framework in which to state what such a driver would be.
- *Typed languages atop combinators.* The encoding of typed languages on top of combinatory logic, particularly in the presence of intensional features, can permit new kinds of type safety for second- and higher-order values. The Hoon language for the Nock combinator calculus exemplifies this but the space has not been extensively explored.

While SKI-on-SKI metacircular interpretation is hardly efficient, the development of a toolkit of reusable components and abstractions for SKI-based metacircular interpretation affords a promising structure for the future exploration of combinator-based computing.

Data availability statement

A repository containing the scripts and harnesses used to produce the results in this paper is available at https://github.com/sigilante/artifact-metacircular-ski. Every count in this paper is reproducible from the harnesses in the artifact repository: `tower_harness.py` for T0, T1 and T2, `scry_harness.py` and `scry_namespace.py` for the blocking interpreters, and `scry_paths.py` for the compound-path equality. They require only the Python package `aviary-kernel`, and the two notebooks recompute the 618-atom interpreter, the 43-atom unquoter and the 504,930-contraction self-application from source.

References

Harold Abelson, Gerald Jay Sussman, and Julie Sussman. 1996. *Structure and Interpretation of Computer Programs* (2nd ed.). MIT Press, Cambridge, MA.

Nada Amin and Tiark Rompf. 2018. Collapsing Towers of Interpreters. *Proceedings of the ACM on Programming Languages* 2, POPL (Jan. 2018), 33:1–33:33. doi:10.1145/3158140

Kenichi Asai, Satoshi Matsuoka, and Akinori Yonezawa. 1996. Duplication and Partial Evaluation: For a Better Understanding of Reflective Languages. *Lisp and Symbolic Computation* 9, 2–3 (1996), 203–241.

Henk P. Barendregt. 1991. Self-Interpretation in Lambda Calculus. *Journal of Functional Programming* 1, 2 (1991), 229–233.

Alessandro Berarducci and Corrado Böhm. 1993. A Self-Interpreter of Lambda Calculus Having a Normal Form. In *Computer Science Logic (CSL '92) (Lecture Notes in Computer Science, Vol. 702)*. Springer, 85–99.

Matt Brown and Jens Palsberg. 2016. Breaking Through the Normalization Barrier: A Self-Interpreter for F-omega. In *Proceedings of the 43rd ACM SIGPLAN Symposium on Principles of Programming Languages (POPL 2016)*. 5–17.

N. E. Davis. 2025. Metacircular Virtualization and Practical Nock Interpretation. *Urbit Systems Technical Journal* 2, 1 (2025). https://ustj.urbit.org/article/v02-i01/metacircular-virtualization-and-practical-nock-interpretation

Yoshihiko Futamura. 1971. Partial Evaluation of Computation Process: An Approach to a Compiler-Compiler. *Systems, Computers, Controls* 2, 5 (1971), 45–50. Reprinted in Higher-Order and Symbolic Computation 12(4):381–391, 1999.

Jr. Guy L. Steele and Gerald Jay Sussman. 1978. *The Art of the Interpreter, or the Modularity Complex (Parts Zero, One, and Two)*. Technical Report AI Memo 453. MIT Artificial Intelligence Laboratory.

Jan Martin Jansen. 2013. Programming in the λ-Calculus: From Church to Scott and Back. In *The Beauty of Functional Code*. Lecture Notes in Computer Science, Vol. 8106. Springer, 168–180.

Barry Jay. 2016. Programs as Data Structures in λSF-Calculus. *Electronic Notes in Theoretical Computer Science* 325 (2016), 221–236.

Barry Jay. 2018. Self-Quotation in a Typed, Intensional Lambda-Calculus. *Electronic Notes in Theoretical Computer Science* 336 (2018), 207–222. MFPS XXXIII, 2017.

Barry Jay and Thomas Given-Wilson. 2011. A Combinatory Account of Internal Structure. *Journal of Symbolic Logic* 76, 3 (2011), 807–826.

Barry Jay and Jens Palsberg. 2011. Typed Self-Interpretation by Pattern Matching. In *Proceedings of the 16th ACM SIGPLAN International Conference on Functional Programming (ICFP 2011)*. 247–258.

Neil D. Jones. 1996. What Not to Do When Writing an Interpreter for Specialisation. In *Partial Evaluation: International Seminar, Dagstuhl Castle, Germany, February 1996*. Lecture Notes in Computer Science, Vol. 1110. Springer, 216–234.

Neil D. Jones, Carsten K. Gomard, and Peter Sestoft. 1993. *Partial Evaluation and Automatic Program Generation*. Prentice Hall International.

G. A. Kavvos. 2017. *On the Semantics of Intensionality and Intensional Recursion*. Ph. D. Dissertation. University of Oxford. arXiv:1712.09302.

Stephen C. Kleene. 1936. General Recursive Functions of Natural Numbers. *Math. Ann.* 112 (1936), 727–742.

John McCarthy. 1960. Recursive Functions of Symbolic Expressions and Their Computation by Machine, Part I. *Commun. ACM* 3, 4 (1960), 184–195.

John McCarthy, Paul W. Abrahams, Daniel J. Edwards, Timothy P. Hart, and Michael I. Levin. 1962. *LISP 1.5 Programmer's Manual*. MIT Press, Cambridge, MA.

Torben Æ. Mogensen. 1992. Efficient Self-Interpretation in Lambda Calculus. *Journal of Functional Programming* 2, 3 (1992), 345–364.

John C. Reynolds. 1972. Definitional Interpreters for Higher-Order Programming Languages. In *Proceedings of the ACM Annual Conference*. 717–740. Reprinted in Higher-Order and Symbolic Computation 11(4):363–397, 1998.

Dana S. Scott. 1962. A System of Functional Abstraction. (1962). Lectures delivered at the University of California, Berkeley, 1962/63; unpublished manuscript, Stanford University, 1963.

Brian Cantwell Smith. 1984. Reflection and Semantics in Lisp. In *Proceedings of the 11th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL '84)*. 23–35.

John Tromp. 2006. Binary Lambda Calculus and Combinatory Logic. In *Kolmogorov Complexity and Applications (Dagstuhl Seminar Proceedings, Vol. 06051)*. Schloss Dagstuhl.

Alan M. Turing. 1936. On Computable Numbers, with an Application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society* 42 (1936), 230–265.

Mitchell Wand and Daniel P. Friedman. 1988. The Mystery of the Tower Revealed: A Nonreflective Description of the Reflective Tower. *Lisp and Symbolic Computation* 1, 1 (1988), 11–38.

Curtis Yarvin. 2018. Nock 4K. Tlon Corporation, https://docs.urbit.org/language/nock/reference/definition. Accessed 2024-02-20.

A The artifact

All results were produced using the Python package `aviary-kernel`, version 0.1.0.

The terms of Figures 4 and 5 are here written out in full. Application is juxtaposition and associates to the left; parentheses appear only where association is to the right; line breaks fall at spaces and carry no meaning. Both are exactly what bracket abstraction produces from the named definitions, so their atom counts are algorithm-relative in the sense of §4. Each is a fixpoint of its generator, and thus they begin with the same fourteen atoms, which is Y: `S (K (S I I)) (S (S (K S) K) (K (S I I)))`.

`UQ`, the unquoter, 43 atoms:

```
S (K (S I I)) (S (S (K S) K) (K (S I I))) (S (K (S (S (S I (K S)) (K K))
(K I)))) (S (K K) (S (S (K S) (S (K (S (K S))) (S (K K)))) K)))
```

`whnfF`, the interpreter, 618 atoms:

```
S (K (S I I)) (S (S (K S) K) (K (S I I))) (S (K (S (S (K S) (S (K K) (S I (K
K)))))) (S (K K) (S (K (S (S (K S) (S (K K) (S (S (S (S (S (K (S I I)) (S
(S (K S) K) (K (S I I))) (S (K (S (S (K S) (S (S (K S) (S (S (K S) (S (S (K
S) K) (K (S (K (S (K K))) (S (K (S (K K))) (S (K (S I)) K)))))) (K (S (K K)
(S (K (S (K K))) (S (K (S I)) K)))))) (K (S (K K) (S (K K) (S (K (S I))
K)))))))) (S (K K) (S (S (K S) (S (K K) (S (K S) (S (K (S (K S))) (S (K
K)))))) (K (S (K K) (S (K (S (S (K (S (K K))) (S (S (K S) (S (K K) (S (K S)
(S (K (S I)) K)))) (K K))))) K))))))) (K K)) (K (S (S I (K K)) (K (S (K (S
(S I (K K)))) (S (K K) (S (K (S (K (S (S I (K K)))))) (S (K (S (K K))) (S (K
(S (K (S (K (S (K (S (K K) (S (K (S I)) K))))))))) (S (K (S (K (S (K (S (K
(S I I)) (S (S (K S) K) (K (S I I))) (S (K (S (S (K S) (S (K (S I)) K)))) (S
(K (S (K K))) (S (S (K S) (S (K K) (S (K S) K))) (K (S (K (S (K K))) (S (K
(S (K K))) (S (K (S (K K))) (S (S (K S) (S (K K) (S (K S) (S (K (S I)) K))))
(K K)))))))))))))) (S (S (K S) (S (K K) (S (K S) (S (K (S (K (S (K (S (K
K))) (S (K (S (K K))) (S (K (S (K K))) (S (S (K S) (S (K K) (S (K S) (S (K
(S I)) K)))) (K K))))))) (S (S (K S) (S (K K) (S (K S) (S (K (S (K (S (K (S
I)) (S (S (K S) K) (K (S I I))) (S (K (S (S (K S) (S (K (S I)) K)))) (S (K
(S (K K))) (S (S (K S) (S (K K) (S (K S) K))) (K (S (K (S (K K))) (S (K (S
(K K))) (S (K (S (K K))) (S (S (K S) (S (K K) (S (K S) (S (K (S I)) K))))
(K K)))))))))))))))) (K (S (S I (K K)) (K (S (K (S (S I (K
K)))) (S (K K) (S (K K) (S (K (S (K (S (K K) (S (K (S I)) K))))) (S (K (S I
I)) (S (S (K S) K) (K (S I I))) (S (K (S (S (K S) (S (K (S I)) K)))) (S (K
(S (K K))) (S (S (K S) (S (K K) (S (K S) K))) (K (S (K (S (K K))) (S (K (S
(K K))) (S (K (S (K K))) (S (S (K S) (S (K K) (S (K S) (S (K (S I)) K))))
(K K)))))))))))))))) (K (S (S I (K K)) (K (S (K (S (K (S (K K) (S (K (S I))
K))
```